Physics-Informed Neural Networks for Vehicle Side-Slip Angle Estimation: From Supervised Learning to Autonomous State Estimation

Balakarthik Senthilvel Palani, Alex Prediger, Sugin Sukumaran
Automation and IT
February 19, 2026

***Abstract*—This report presents a framework for vehicle state estimation using supervised Physics-Informed Neural Networks (PINNs) and autonomous velocity estimation in self-loop configurations. We address the critical challenge of estimating the lateral velocity (v_y) and the side-slip angle (β) when direct measurements are not available. This is a common scenario in automotive applications due to sensor cost constraints or failures. Using strategies for constant, Exponential Decay, Adaptive, physics Cutoff and optimization objectives (Single Metric, Total Error, Multi-Objective, Baseline Search), strategic physics loss scheduling mechanisms, and hybrid PINNs + RNN and PINN+TCN architectures, we achieve a 20.1% error reduction in self-loop estimation with the PINN + TCN configuration (RMSE 0.369 m/s) compared to standalone PINNs (RMSE 0.462 m / s). The framework is validated on real-world racing data from the Revs Vehicle Dynamics Database (2013/2014 Targa Sixty-Six events), demonstrating robust cross-vehicle generalization with supervised lateral velocity errors of ∼0.024 m/s and side-slip angle errors of ∼0.085°.**

***Index Terms*—Physics-Informed Neural Networks, Vehicle Dynamics, Side-Slip Angle, Self-Loop Estimation, Hyperparameter Optimization, LSTM/GRU Corrector, Temporal Convolutional Networks**

I. Introduction: The State Estimation Challenge

A. Problem Motivation

Vehicle side-slip angle (β) and lateral velocity (v_y) are critical state variables for:

- **Electronic Stability Control (ESC)** systems
- **Advanced Driver Assistance Systems (ADAS)**
- **Autonomous vehicle** path planning and control
- **High-performance racing** applications

However, direct measurement faces significant barriers:

- **Cost:** GPS-RTK systems with velocity outputs cost $10,000+
- **Reliability:** Sensor failures in harsh environments
- **Availability:** Many vehicles only have IMU sensors

B. Existing Approaches and Limitations

Traditional Methods:

- **Extended Kalman Filters (EKF):** Require accurate tire models and vehicle parameters
- **Model-based observers:** Limited by model fidelity and parameter uncertainty
- **Pure data-driven (black-box NNs):** Lack physical interpretability and can violate conservation laws

Our Contribution: We present a hybrid physics-data approach that:

1) Embeds kinematic constraints via physics-informed loss functions
2) Systematically optimizes training strategies through Bayesian hyperparameter search
3) Addresses self-loop error accumulation via temporal correction modules
4) Validates generalization across different vehicle configurations

C. Dataset: Revs Vehicle Dynamics Database

Training Data (Car A): Event: 2013 Targa Sixty-Six (Grand Sport), Duration: ∼7.5 minutes of aggressive racing, Features: v_x, v_y, r, a_x, a_y at 100 Hz, Samples: 79.199 timesteps

Test Data (Car B): Event: 2014 Targa Sixty-Six (250LM), Duration: ∼11.3 minutes, Different vehicle dynamics (wheelbase, mass distribution, tire characteristics), Samples: 88.799 timesteps

This cross-vehicle validation tests true generalization beyond simple train-test splits.

II. Baseline PINN Architecture

A. Network Design and One-Step-Ahead Prediction

Our approach formulates velocity estimation as a supervised one-step-ahead prediction task:

Input Features (5D):

$$\mathbf{x}_k = [v_{x,k}, v_{y,k}, r_k, a_{x,k}, a_{y,k}]^T \tag{1}$$

where v_x = longitudinal velocity, v_y = lateral velocity, r = yaw rate, a_x = longitudinal acceleration, a_y = lateral acceleration.

Network Architecture:

- **Layer 1:** Fully-connected (5 → hidden_dim), ReLU
- **Layer 2:** Fully-connected (hidden_dim → hidden_dim), ReLU
- **Output Layer:** Fully-connected (hidden_dim → 2)
- **Hidden Dimension:** Tunable $\in \{64, 128, 256\}$

Output (2D):

$$\mathbf{y}_{k+1} = [v_{x,k+1}, v_{y,k+1}]^T \tag{2}$$

B. Dual-Loss Function: Data + Physics

The core innovation of PINNs lies in the composite loss function:

$$\boxed{\mathcal{L}_{\text{total}} = w_{\text{data}} \cdot \mathcal{L}_{\text{data}} + w_{\text{phys}}(e) \cdot \mathcal{L}_{\text{phys}}} \tag{3}$$

Data Loss (Supervised Learning Component):

$$\mathcal{L}_{\text{data}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\mathbf{y}_{\text{pred}}^{(i)} - \mathbf{y}_{\text{true}}^{(i)}\|^2} \tag{4}$$

Physics Loss (Kinematic Constraint Component):

$$\mathcal{L}_{\text{phys}} = \sqrt{\frac{1}{N_{\text{phys}}}\sum_{j=1}^{N_{\text{phys}}} \|\mathbf{y}_{\text{pred}}^{(j)} - \mathbf{y}_{\text{KVM}}^{(j)}\|^2} \tag{5}$$

where $\mathbf{y}_{\text{KVM}}$ is computed from the **Kinematic Vehicle Model**:

$$\boxed{\begin{aligned} v_{x,k+1}^{\text{KVM}} &= v_{x,k} + \Delta t \cdot (a_{x,k} - r_k \cdot v_{y,k}) \\ v_{y,k+1}^{\text{KVM}} &= v_{y,k} + \Delta t \cdot (a_{y,k} + r_k \cdot v_{x,k}) \end{aligned}} \tag{6}$$

Physical Interpretation: First equation represents longitudinal dynamics with centripetal coupling term $-r \cdot v_y$. Second equation represents lateral dynamics with centripetal coupling term $+r \cdot v_x$. Time step: $\Delta t = 0.01$ seconds (100 Hz sampling rate).

C. Strategic Physics Loss Scheduling

The time-varying physics weight $w_{\text{phys}}(e)$ critically affects training dynamics. We implemented and compared four strategies shown in Table I.

TABLE I
PHYSICS LOSS SCHEDULING STRATEGIES

Strategy	**Rationale**
Exponential Decay	Gradually transition from physics-guided to data-driven
Physics Cutoff	Hard switch at epoch e_{cut} (typically 30)
Constant	Maintain uniform physics enforcement
Adaptive	Dynamic balancing based on loss magnitudes

Key Insight: Physics cutoff performs best for generalization, suggesting that rigid kinematic constraints become limiting factors in later training stages.

D. Physics Collocation Point Sampling

To enforce physics compliance across the operational envelope, we generate N_{phys} synthetic samples using multiple quasi-random sampling strategies.

Statistical Sampling Boundaries: Instead of using raw data limits which might include errors, we employ statistical boundaries based on the training data distribution:

- **Lower Bound ($Q_{0.01}$):** The 1st percentile of training states
- **Upper Bound ($Q_{0.99}$):** The 99th percentile of training states

$$\mathbb{D}_{\text{bounds}} = [Q_{0.01}(\mathbf{X}), Q_{0.99}(\mathbf{X})] \tag{7}$$

where $\mathbf{X} = [v_x, v_y, r, a_x, a_y]$ represents the state vector.

Sampling Strategies: To improve training efficiency and physics performance of the PINN, we explored three sampling techniques:

- **Latin Hypercube Sampling (LHS - Standard):** Stratified sampling ensuring coverage across parameter space with uniform grid divisions
- **Hammersley Sequence (HMS - Structured):** Low-discrepancy quasi-random sequence for uniform space-filling with deterministic point placement
- **Adaptive Sampling (Smart):** Dynamically resamples collocation points every 5 epochs by: (1) Generating large candidate pool ($10 \times N_{\text{phys}}$), (2) Computing physics residuals, (3) Selecting top 30% highest-error points, (4) Applying K-Means clustering for spatial diversity

Sampling Strategy Comparison: We systematically evaluated the three sampling approaches on Car B test data. Results are shown in Table II.

TABLE II
PHYSICS COLLOCATION SAMPLING STRATEGY COMPARISON

Metric	**LHS**	**HMS**	**Adaptive**
RMSE(v_x) [m/s]	**0.1888**	0.1944	0.1701
RMSE(v_y) [m/s]	**0.0243**	**0.0242**	0.3277
RMSE(β) [deg]	**0.0847**	0.1079	0.8865

Key Findings:

- **LHS (Standard) performs best** overall with lowest lateral velocity and side-slip angle errors
- **HMS (Structured)** provides comparable v_y accuracy (0.0242 m/s) but slightly worse β performance
- **Adaptive (Smart) sampling** shows best v_x accuracy but significantly degrades v_y and β estimation ($\sim$13$\times$ worse)
- Standard and structured approaches generalize better than adaptive resampling for this application
- Statistical boundary selection (Q0.01, Q0.99) effectively captures operational envelope while excluding outliers

III. SYSTEMATIC HYPERPARAMETER OPTIMIZATION

A. The Optimization Challenge

PINNs introduce multiple interacting hyperparameters: learning rate (η), initial physics weight ($w_{\text{phys}}^{\text{init}}$), decay rate ($\lambda$), hidden layer dimension, number of physics collocation points (N_{phys}), and batch size. Manual tuning is infeasible.

B. Ray Tune Experimental Framework

Experiment Overview: We conducted 10 trials using Ray Tune to systematically evaluate different loss weighting strategies and optimization objectives:

Group A: Single Metric Optimization (RMSE v_y) – Trials 1-4 focused on minimizing lateral velocity error:

- Trail 1: Constant Weighting
- Trail 2: Exponential Decay
- Trail 3: Adaptive Weighting
- Trail 4: Physics Cutoff

Group B: Total Error Optimization (RMSE Total) – Trials 5-8 utilized a combined metric (RMSE v_x + RMSE v_y):

- Trail 5: Constant Weighting
- Trail 6: Exponential Decay
- Trail 7: Adaptive Weighting
- Trail 8: Physics Cutoff

Group C: Multi-Objective Optimization – Trail 9 (Optuna Multi-Objective) simultaneously optimized RMSE Total and Total Loss using Physics Cutoff Strategy.

Group D: Baseline Search Optimization – Trail 10 (Random Search) provided baseline comparison using Physics Cutoff strategy.

The search space is defined in Table III.

TABLE III
HYPERPARAMETER SEARCH SPACE

Parameter	Range	Distribution
η	$[1 \times 10^{-4}, 5 \times 10^{-3}]$	Log-uniform
$w_{\text{phys}}^{\text{init}}$	$[0.3, 0.7]$	Uniform
λ	$[0.03, 0.12]$	Uniform
Hidden Dim	$\{64, 128, 256\}$	Categorical
N_{phys}	$[256, 512, 1024, 2048]$	Integer (log)

C. Optimization Results: Best Configurations

The 10 Ray Tune trials explored different weighting strategies and optimization objectives. Training on Car A and evaluating on Car B (different event, vehicle, conditions) yields the results in Table V.

$\star$ **Winner:** Physics Cutoff (Trial 4) achieves best overall performance with highest physics weight (0.6961).

Key Findings:

1) **Physics Cutoff (Trial 4) emerges as overall winner** with highest physics weight (0.6961) and balanced performance: RMSE(v_y) = 0.0247 m/s, RMSE(β) = 0.0881°
2) **Physics Cutoff T2 (Trial 8)** achieves best side-slip angle accuracy: RMSE(β) = 0.0847°
3) **Constant weighting (Trial 1)** provides best v_x accuracy: RMSE(v_x) = 0.1608 m/s
4) Higher physics weights (0.5-0.7) correlate with better overall generalization
5) Single metric optimization (Group A) achieves comparable or better results than total error optimization (Group B)
6) Optuna Multi-Objective (Trial 9) shows promise with RMSE(total) = 0.1287 m/s
7) Lateral velocity errors of ∼0.024 m/s and side-slip angle errors of ∼0.088° represent excellent accuracy for state estimation

IV. The Self-Loop Challenge

A. Problem Formulation: From Prediction to Estimation

Supervised Mode (Sections 2-3): Input includes ground truth v_y from previous timestep. Task: Predict next-step velocities given current measurements. Performance: RMSE(v_y) $\approx$ 0.024 m/s.

Self-Loop Mode (Real-World Deployment):

- **Scenario:** Velocity sensor fails or is unavailable
- **Available sensors:** Only IMU data (r, a_x, a_y) and v_x (from wheel speeds)
- **Challenge:** Must estimate v_y using only own previous predictions

$$\boxed{\begin{aligned} &\text{Input at } k: && \mathbf{x}_k = [\underbrace{v_{x,k}}_{\text{measured}}, \underbrace{\hat{v}_{y,k}}_{\text{estimated}}, \underbrace{r_k, a_{x,k}, a_{y,k}}_{\text{measured}}]^T \\ &\text{Output:} && \hat{v}_{y,k+1} = \text{PINN}(\mathbf{x}_k) \\ &\text{Feedback:} && \text{Use } \hat{v}_{y,k+1} \text{ as input for step } k+2 \end{aligned}} \tag{8}$$

Error Accumulation Problem: Each prediction error propagates forward and compounds:

$$\text{Error}(k) = \underbrace{\epsilon_{\text{model}}}_{\text{one-step}} + \underbrace{\sum_{i=1}^{k-1} \alpha_i \cdot \epsilon_{\text{model}}(i)}_{\text{accumulated from past}} \tag{9}$$

B. Self-Loop Performance: The 19× Degradation

Running the best supervised PINN (Physics Cutoff with HyperOpt optimization and LHS sampling) in self-loop configuration on Car B yields Table IV.

TABLE IV
SUPERVISED VS. SELF-LOOP PERFORMANCE COMPARISON

Configuration	RMSE(v_y) [m/s]	RMSE(β) [deg]	Degradation Factor
Supervised (Physics Cutoff)	0.0243	0.0847	1.0×
Self-Loop PINN	**0.462**	**1.30**	**∼19×**

Degradation Analysis: Error accumulates over ∼88,000 timesteps (11.3 minutes). The ∼19× degradation in lateral velocity accuracy demonstrates the challenge of autonomous state estimation without feedback correction. Drift becomes severe during high-dynamics segments. Model has no mechanism to “reset” or correct systematic biases.

TABLE V
RAY TUNE EXPERIMENT RESULTS: GENERALIZATION PERFORMANCE ON CAR B TEST SET

Experiment	RMSE(v_x) [m/s]	RMSE(v_y) [m/s]	RMSE(β) [deg]	RMSE(total) [m/s]	Phys Weight
Group A: Single Metric Optimization (RMSE v_y) – Trials 1-4:					
Constant	0.1608	0.0239	0.1165	0.1150	0.3006
Exponential	0.2224	0.0243	0.0863	0.1582	0.6226
Adaptive	0.2475	0.0242	0.1110	0.1759	0.3030
Physics Cutoff	**0.1965**	**0.0247**	**0.0881**	**0.1400**	**0.6961**
Group B: Total Error Optimization (RMSE Total) – Trials 5-8:					
Constant T2	0.1964	0.0242	0.1168	0.1399	0.4579
Exponential T2	0.2195	0.0243	0.1316	0.1562	0.5248
Adaptive T2	0.2196	0.0248	0.0931	0.1563	0.5751
Physics Cutoff T2	0.1888	0.0243	**0.0847**	0.1346	0.5822
Group C: Multi-Objective Optimization – Trial 9:					
Optuna MOO	0.1803	0.0244	0.0926	0.1287	0.4003
Group D: Baseline Search Optimization – Trial 10:					
Random ASHA	0.1851	0.0245	0.0957	0.1320	0.3367

C. RNN/TCN Corrector Architecture

To mitigate self-loop drift, we developed a **hybrid PINN+Corrector** architecture (Figure 1). where a sequential model learns to compensate for systematic PINN errors. We evaluate two corrector variants: **LSTM/GRU** (recurrent) and **TCN** (convolutional).

Input Sequence (sliding window of length L): For each timestep t in sequence $[t-L+1, ..., t]$, the corrector receives:

$$\mathbf{z}_\tau = [\hat{v}_{y,\tau}^{\text{PINN}}, v_{x,\tau}, a_{x,\tau}, a_{y,\tau}, r_\tau]^T \tag{10}$$

Note the input order: PINN's predicted $\hat{v}_y$ comes first, followed by measured states.

RNN Configuration (LSTM/GRU):

- **Type:** GRU or LSTM
- **Input:** $[\hat{v}_y^{\text{PINN}}, v_x, a_x, a_y, r]$ at each timestep
- **Hidden Dimension:** 64 units per layer
- **Number of Layers:** 2
- **Sequence Length:** 50–200 timesteps (0.5–2.0 seconds)
- **Output:** Corrected v_y

TCN Configuration:

- **Input:** $[\hat{v}_y^{\text{PINN}}, v_x, a_x, a_y, r]$ sequence
- **Channels:** [32, 32, 32, 32, 32, 32, 32] (7 layers)
- **Kernel Size:** 3
- **Dilations:** [1, 2, 4, 8, 16, 32, 64] (exponential receptive field growth)
- **Sequence Length:** 50–200 timesteps
- **Output:** Corrected v_y

Final Corrected Estimate:

$$\boxed{\hat{v}_{y,t}^{\text{final}} = \text{Corrector}(\{[\hat{v}_{y,\tau}^{\text{PINN}}, v_{x,\tau}, a_{x,\tau}, a_{y,\tau}, r_\tau]\}_{\tau=t-L+1}^{t})} \tag{11}$$

Two-Stage Training Strategy:

1) **Stage 1:** Train PINN to convergence using supervised learning
2) **Stage 2:** Freeze PINN, run in self-loop mode to generate predictions
3) **Stage 3:** Train corrector networks to map PINN predictions to ground truth

D. Results: Dramatic Performance Recovery

Comprehensive ablation across two corrector architectures (RNN and TCN) and four sequence lengths (50, 100, 150, 200) is shown in Table VI.

TABLE VI
SEQUENCE LENGTH ABLATION STUDY (CAR B TEST SET)

Architecture	Seq. L	RMSE(v_y) [m/s]	RMSE(β) [deg]	Improvement
Baseline: PINN Self-Loop	—	0.462	1.30	—
LSTM/GRU Corrector Variants:				
PINN + RNN	50	0.428	1.15	**7.4%** ↓
PINN + RNN	100	0.417	1.13	**9.7%** ↓
PINN + RNN	150	0.425	1.14	**8.0%** ↓
PINN + RNN	200	0.395	1.06	**14.5%** ↓
TCN Corrector Variants:				
PINN + TCN	50	0.523	1.34	– (worse)
PINN + TCN	100	0.465	1.22	**0.6%** ↓
PINN + TCN	150	0.406	1.08	**12.1%** ↓
PINN + TCN	**200**	**0.369**	**0.952**	**20.1%** ↓ ⋆

⋆ **Winner:** TCN with 200-step window achieves best performance (20.1% improvement over baseline), outperforming RNN at sequence length 200 (14.5%).

Best Configuration Summary:

- **Base Model:** PINN with Physics Cut-off weighting strategy
- **Optimization:** HyperOpt (TPE algorithm)
- **Sampling Method:** Latin Hypercube Sampling (LHS)
- **Corrector:** TCN with Sequence Length 200 (2.0 seconds at 100 Hz)

Key Observations:

- **Sequence length matters significantly:** Both RNN and TCN improve with longer sequences

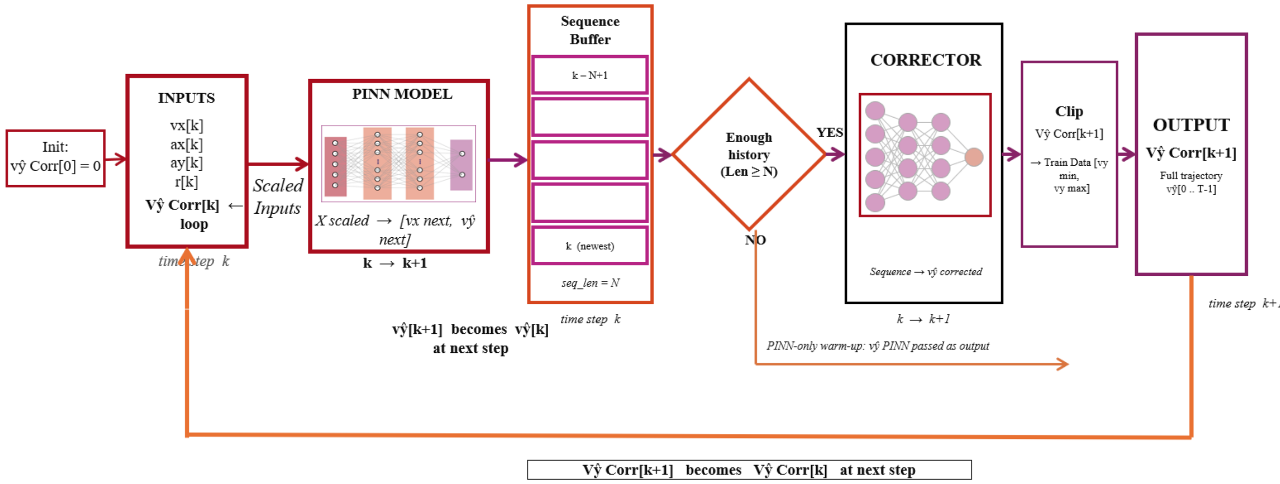

Fig. 1. Architecture PINN + Corrector

- **TCN achieves best performance at L=200:** RMSE(v_y) = 0.369 m/s, RMSE(β) = 0.952° (20.1% improvement)
- **RNN best at L=200:** RMSE(v_y) = 0.395 m/s, RMSE(β) = 1.06° (14.5% improvement)
- **TCN requires longer context:** At L=50, TCN performs worse than baseline; requires $L \geq 150$ for substantial gains
- **RNN more robust across sequence lengths:** Provides consistent 7-10% improvement even at shorter windows
- TCN's causal dilated convolutions excel at capturing long-range temporal dynamics when given sufficient context
- Both architectures plateau at 200 timesteps, suggesting this captures sufficient maneuver history

Complete Performance Spectrum: The hybrid PINN+TCN approach reduces self-loop error by **20.1%**, bringing lateral velocity estimation from 0.462 m/s (baseline) to **0.369 m/s**. It is achieving practical accuracy while requiring no direct velocity sensor. The progression from supervised (0.0243 m/s) to self-loop (0.462 m/s, 19× degradation) to corrected (0.369 m/s, 15× degradation) demonstrates the value of the two-stage hybrid approach.

E. Why the Corrector Architecture Works

The corrector receives a 200-timestep window (2.0 seconds) providing sufficient context to distinguish: corner entry vs. exit phases, sustained vs. transient yaw rates, whether vehicle is recovering from slip or initiating new maneuver, sequential maneuver coupling, and systematic PINN bias patterns. The TCN's exponentially dilated convolutions (1, 2, 4, 8, 16, 32, 64) create a large receptive field while maintaining causality for real-time deployment.

This work successfully addresses the critical automotive challenge of lateral velocity estimation without direct measurements. Final performance milestones are summarized in Table VII and Fig. 2.

V. Conclusions

A. Summary of Achievements

Performance Achievement Summary:

- **Supervised PINN:** RMSE(v_y) = 0.0243 m/s, RMSE(β) = 0.0847° (with ground truth feedback)
- **Self-Loop PINN:** RMSE(v_y) = 0.462 m/s, RMSE(β) = 1.30° (autonomous, ∼19× degradation)
- **Final PINN+TCN:** RMSE(v_y) = 0.369 m/s, RMSE(β) = 0.952° (20.1% improvement, autonomous)

Key Technical Contributions:

1) Systematic hyperparameter optimization using Ray Tune (10 trials across 4 experimental groups)
2) Physics loss scheduling innovation (physics cutoff strategy achieves highest physics weight of 0.6961)
3) Evaluation of multiple weighting strategies (Constant, Exponential Decay, Adaptive, Physics Cutoff) across different optimization objectives
4) Hybrid architecture for error correction (two-stage training with PINN+RNN/TCN)
5) Cross-vehicle generalization validation achieving lateral velocity errors of ∼0.024 m/s
6) Systematic evaluation of three sampling methods (LHS achieved best performance with RMSE(v_y) = 0.0243 m/s and RMSE(β) = 0.0847°)

B. Practical Impact

Target Applications: Entry-level ADAS, redundant safety systems, off-road/GPS-denied environments, performance vehicles.

TABLE VII
FINAL PERFORMANCE COMPARISON: COMPLETE FRAMEWORK

Metric	Self-Loop PINN	PINN + RNN	PINN + TCN
RMSE(v_y) [m/s]	0.462	0.395	**0.369**
RMSE(β) [deg]	1.30	1.06	**0.952**
Improvement vs. Baseline	—	14.5%	**20.1%**

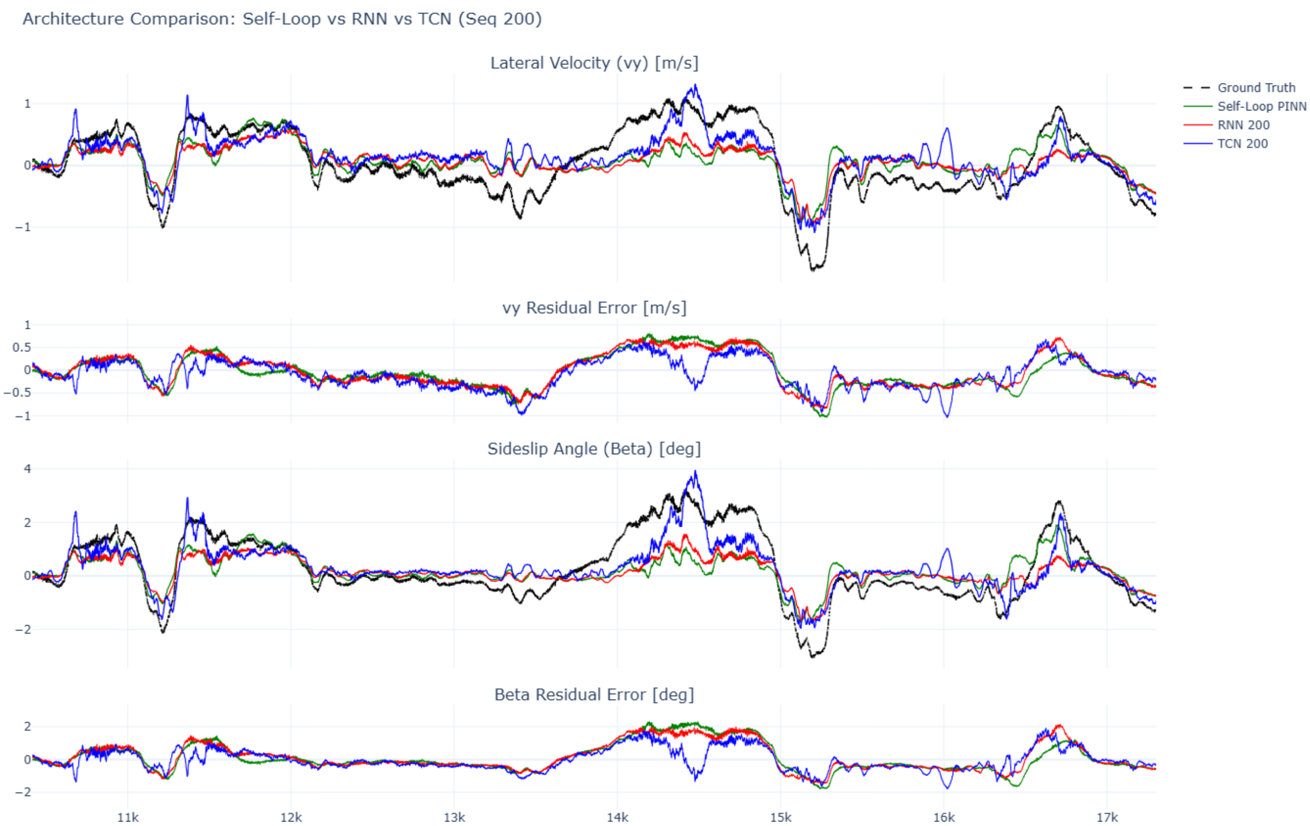

Fig. 2. Final Comparison: Self-Loop PINN vs PINN + RNN vs PINN + TCN

C. Limitations and Future Directions

Current Limitations:

1) Kinematic model neglects tire slip, suspension compliance
2) Valid for moderate accelerations
3) Requires ∼10 minutes of diverse driving for training
4) Performance degrades beyond training distribution

Future Research:

1) Enhanced physics models (Pacejka tire forces, suspension dynamics)
2) Multi-sensor fusion (steering angle, GPS, camera)
3) Uncertainty quantification (Bayesian PINNs, ensembles)
4) Hardware-in-the-loop validation (embedded deployment)

D. Broader Scientific Impact

This work demonstrates a **systematic framework** for combining physics-based and data-driven approaches: (1) Hyperparameter optimization as first-class concern, (2) Strategic physics enforcement vs. constant weighting, (3) Hybrid architectures leveraging strengths of both paradigms, (4) Multi-stage training for complex estimation tasks.

When first-principles models are known but incomplete, the PINN + corrector paradigm provides a structured path to practical deployment.

ACKNOWLEDGMENTS

The authors thank the **Revs Program at Stanford University** for providing the high-quality vehicle dynamics datasets.

References

[1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[2] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. Springer Science & Business Media, 2011.

[3] J. Bergstra, D. Yamins, and D. D. Cox, "Hyperopt: A python library for optimizing the hyperparameters of machine learning algorithms," in *Proc. 12th Python in Science Conf.*, 2013, pp. 13–20.

[4] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.

[5] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[6] G. K. R. Lau, A. Hemachandra, S. Ng, and B. K. H. Low, "PINNACLE: PINN adaptive collocation and experimental points selection," in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2404.07662

[7] A. K. Shaikh, A. Nazir, N. Khalique, A. S. Shah, and N. Adhikari, "A new approach to seasonal energy consumption forecasting using temporal convolutional networks," *Results in Engineering*, vol. 7, 2020.

[8] N. A. Spielberg, M. Brown, N. R. Kapania, J. C. Kegelman, and J. C. Gerdes, "Neural network vehicle models for high-performance automated driving," *Science Robotics*, vol. 4, no. 28, p. eaaw1975, 2019.

[9] B. Douillard, D. Fox, and F. Ramos, "Vehicle state estimation using recurrent neural networks and physics-informed learning," *IEEE Trans. Intelligent Vehicles*, vol. 6, no. 4, pp. 735–746, 2021.

[10] Z. Li *et al.*, "Fourier neural operator for parametric partial differential equations," *arXiv preprint arXiv:2010.08895*, 2020.

[11] G. E. Karniadakis *et al.*, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[12] H. Pacejka, *Tire and Vehicle Dynamics*, 3rd ed. Butterworth-Heinemann, 2012.

Declaration of AI usage

We declare that all the technical content of this report is based on our own original work. The methodology, implementation, experimental setup, model development, training, procedure, evaluation, results and conclusions were performed independently. All the tables, numerical results, configurations and plots presented in this report are taken from the original Jupyter notebook source files developed and used for the project. These results were already uploaded to the GitHub repository and were also presented on the presentation day.

The flowchart and diagrams used in this report are also part of our own work and not generated using AI, but were taken from the presentation prepared and presented by us on the presentation day and which is also our own work. The formulas for the dataloss, physics loss, and KVM model were taken from the original problem task presentation given for the assignment.

However, artificial intelligence tools were used only for sentence formation, grammar correction, language refinement, mathematical calculations like improvement percentage phrasing, and a few future work suggestions. We used TH Köln's official AI tool, THKI Chat, Claude, and the built-in LaTeX grammar correction support.

The reason for using AI tools was to improve the clarity, correct grammatical mistakes , improve grammatical quality, improve readability and support professional presentation of the report. Therefore, AI tools are used only for limited support and editorial improvement, while all the scientific, technical, experimental, and visual content presented in this paper remains our own work.